نظریه زبان‌ها و ماشین‌ها

ترم زمستان ۱۴۰۴-بهار ۱۴۰۵

جلسه چهارم

موضوع این جلسه:

- مرور جلسه پیش: پذیرنده‌ی متناهی قطعی (dfa)
- معرفی پذیرنده‌ی متناهی «غیرقطعی» یا nfa (فصل ۲,۲):
  - تعریف
  - چند مثال
- تبدیل nfa به dfa
- کمینه‌سازی تعداد حالت‌های dfa

مرور جلسات پیش:

مفاهیم الفبا، رشته و زبان را تعریف کردیم.

الفبا	Finite, non-empty set	$\sum = \{a, b\}$
رشته	Finite sequence	$v = abb$
رشته تهی	λ	$\|\lambda\| = 0$
تمام رشته‌ها		$\sum^* = \{\lambda, a, b, ab, aa, bb, abb, ...\}$
زبان	Any subset of $\sum^*$	$L = \{a, abb\}$

یادآوری: پذیرنده متناهی قطعی (Deterministic Finite Accepter → dfa)

مثال:

EXAMPLE 2.2

Example: aab

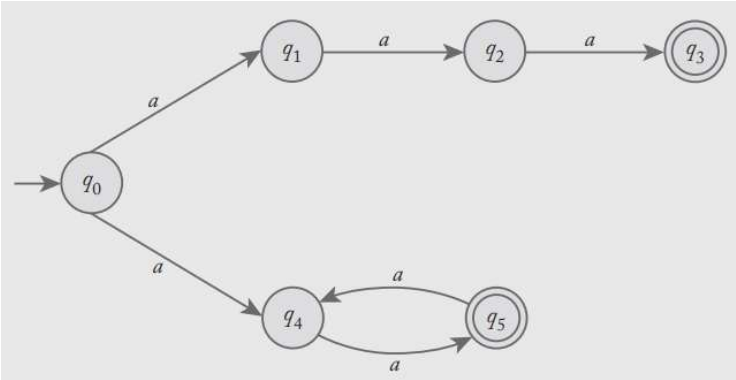
$L = \{a^n b : n \geq 0\}$

معرفی پذیرنده متناهی «غیرقطعی»:

در بررسی پذیرنده‌های متناهی قطعی (dfa)، دیدیم که برای هر state، به ازای هر نماد از الفبا، «دقیقاً یک» گذار تعریف می‌شود. پذیرنده متناهی «غیرقطعی» همان پذیرنده متناهی قطعی است با این تفاوت که این قید را ندارد. یعنی در این پذیرنده‌ها، برای هر state، به ازای هر نماد از الفبا، ممکن است صفر، یک یا چند گذار تعریف شود. به علاوه، با رشته‌ی تهی هم، گذار امکان‌پذیر است.

مثال (EXAMPLE 2.7): گراف زیر، یک پذیرنده‌ی متناهی **غیرقطعی** را نشان می‌دهد چون برای حالت q_0، با دیدن نماد a در ورودی، دو گذار تعریف شده است:

- گذار به q_1
- گذار به q_4



در نتیجه، برای یک رشته، چند مسیرِ ممکن وجود دارد که nfa می‌تواند طی کند مثلاً برای رشته‌ی aa:

$$q_0 \to a \to \begin{cases} \to q_1 \to a \to q_2 \\ \to q_4 \to a \to q_5 = \text{Final state} \end{cases}$$

یک رشته توسط یک nfa پذیرفته می‌شود اگر (حداقل) یک دنباله از گذارها با استفاده از آن رشته وجود داشته باشد که nfa را از حالت اولیه به یک حالت نهایی ببرد.

DEFINITION

A **nondeterministic finite accepter** or **nfa** is defined by the quintuple

ناقطعی

$$M = (Q, \Sigma, \delta, q_0, F),$$

where Q, Σ, q_0, F are defined as for deterministic finite accepters, but δ is different:

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \to 2^Q.$$

EXAMPLE **2.8**

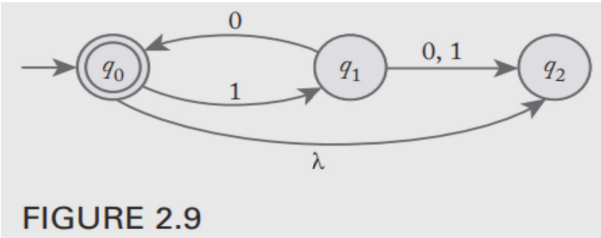


FIGURE 2.9

$\delta(q_0, 1) = \{q_1\}$
$\delta(q_0, 0) = \{\}$
$\delta(q_0, \lambda) = \{q_2\}$
$\delta(q_1, 0) = \{q_0, q_2\}$
$\delta(q_1, 1) = \{q_2\}$

زبان یک nfa: مجموعه‌ی تمام رشته‌هایی که nfa می‌پذیرد.

Extended transition function:

$$\delta : Q \times (\Sigma \cup \{\lambda\}) \to 2^Q.$$
$$\delta^* : Q \times (\Sigma^*) \to 2^Q.$$

If $\delta^*(q_i, w) = Q_j$, then Q_j is the set of all possible states the automaton may be in, having started in state q_i and having read w.

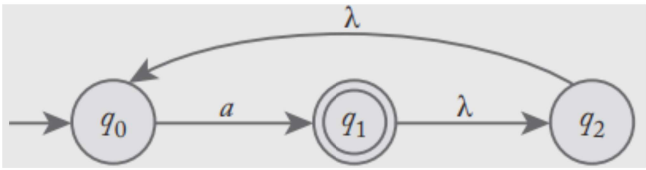
بر این اساس می‌توانیم تعریف دقیق‌تری از زبانی که یک nfa می‌پذیرد داشته باشیم:

For an nfa $M = (Q, \Sigma, \delta, q_0, F)$, language accepted by M is defined as:

$$L(M) = \{w \in \Sigma^* : \delta^*(q_0, w) \cap F \neq \varnothing\}$$

where $\delta^*: Q \times \Sigma^* \to 2^Q$

مثال (EXAMPLE 2.9):



مجموعه حالت‌هایی که nfa از حالت اولیه و پس از دیدن نماد a می‌تواند در آن‌ها باشد:

$$\delta^*(q_0, a) = \{q_1, q_2, q_0\}$$

سوال: چه‌طور مجموعه‌ی حالت‌هایی که nfa پس از دیدن یک رشته از ورودی به آن‌ها خواهد رسید را به‌دست آوریم؟

چالش: وجود گذارهای λ باعث می‌شود که ندانیم روند پیدا کردن حالت‌ها را تا کجا باید ادامه دهیم.

الگوریتم یافتن $\delta^*(q_i, w)$ یعنی به دست آوردن تمام حالت‌هایی که یک nfa با شروع از حالت q_i با رشته‌ی w به آن‌ها می‌رسد.

- با استفاده از الگوریتم یافتن گشت‌های یک گراف (walks)

<u>سوال</u>: آیا محاسبه تمام گشت‌های به طول $|w|$ که از q_i شروع می‌شوند (= گشت‌هایی که تعداد $|w|$ یال دارند و از رأس q_i شروع می‌شوند) کافی است؟

اگر در گراف یک loop با برچسب λ وجود داشته باشد، می‌توان گشت‌های با هر طول دلخواه ایجاد کرد بدون افزایش «طول رشته‌ی پیموده شده».

مثال با شروع از q_1 و با رشته‌ی a در مثال بالا:

walk: $(q_1,q_2), (q_2,q_0), (q_0,q_1), (q_1,q_2) \to \lambda\lambda a\lambda$

string: a

راه حل: برای پیدا کردن مجموعه حالت‌هایی که یک nfa با شروع از حالت q_i با یک رشته‌ی مشخص به آن‌ها خواهد رسید:

کافی است تمام walkهای به طول حداکثر $\Lambda + (1 + \Lambda)\,|w|$ که با رشته‌ی w می‌توان پیمود را بدست آورد با فرض اینکه:

Λ = تعداد یال‌های با برچسب λ در گراف

بخش ۲,۳ – معادل بودن dfa و nfa

سوال: آیا زبانی وجود دارد که برای آن nfa وجود داشته باشد ولی هیچ dfa ای وجود نداشته باشد؟

ادعا: nfa و dfa معادل هستند. یعنی برای هر dfa، (حداقل) یک nfa وجود دارد که همان زبان را می‌پذیرد و برعکس.

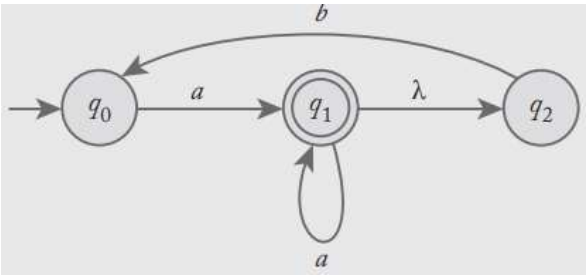
اثبات:

تبدیل dfa به nfa: بنا به تعریف، هر dfa درواقع یک nfa هم هست. در نتیجه این بخش، خود به خود اثبات شده است.

تبدیل nfa به dfa: هدف این است که برای یک nfa، یک dfa بسازیم که معادل آن nfa باشد (یعنی همان زبان را بپذیرد).

تفاوت اصلی dfa و nfa: در dfa، پس از دیدن یک رشته در ورودی، dfa دقیقاً در «یک» حالت خواهد بود اما برای nfa، «مجموعه‌ای» از حالت‌ها وجود دارد که nfa می‌تواند در هر یک از آن‌ها باشد. به بیان دیگر، nfa «همزمان در چند حالت» قرار دارد. برای ساخت dfa معادل، ایده این است که به ازای هر مجموعه از چنین حالت‌هایی، یک حالت در dfa بسازیم.

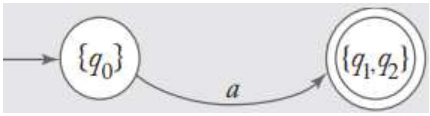
مثال: dfa زیر را در نظر بگیرید.



این dfa، پس دیدن a، می‌تواند در یکی از دو حالت q_1 یا q_2 باشد:

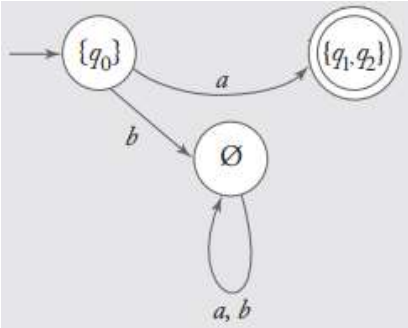
$$q_0 \text{ --- } a \text{ ---} \begin{cases} \rightarrow q_1 \\ \rightarrow q_2 \end{cases}$$

پس در dfa معادل، یک گذار از حالت $\{q_0\}$ به حالتی که اجتماع q_1 و q_2 است قرار می‌دهیم:



در ضمن حالت $\{q_1, q_2\}$ یک حالت نهایی است زیرا ؟

مرحله بعد، بررسی تغییر حالت dfa با نماد b است. چون در nfa هیچ گذاری از q_0 با b وجود ندارد، در dfa یک گذار به حالت «تهی» اضافه می‌کنیم.



تا اینجا، تمام گذارهای ممکن برای حالت $\{q_0\}$ را مشخص کردیم. سپس باید گذارهای خروجی از حالت $\{q_1, q_2\}$ را مشخص کنیم.

ابتدا گذار از حالت $\{q_1, q_2\}$ برای نماد a را تعیین می‌کنیم.

$$\rightarrow q_1 \text{ --- } a \rightarrow q_2$$

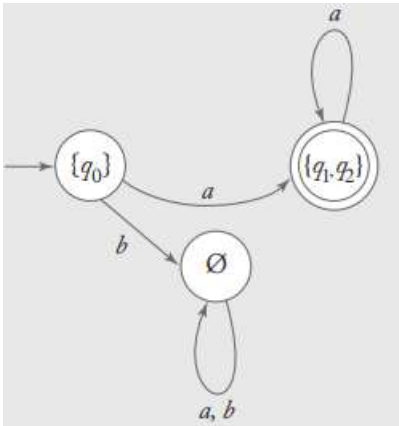
$$q_0 \text{ --- } a \text{ ---}|$$

$$\rightarrow q_2 \text{ --- } a \rightarrow \text{Undefined}$$

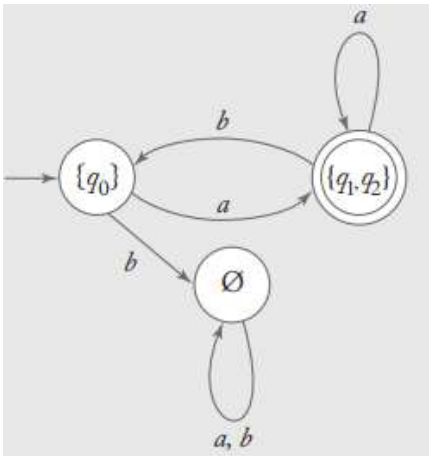
در nfa:

- از q_1 با a یک گذار به q_1 وجود دارد و همچنین سپس با λ به q_2،
- از q_2 هیچ گذاری با a وجود ندارد.

بنابراین، اجتماع گذراها از $\{q_1, q_2\}$ برای a برابر با $\{q_1, q_2\}$ است.



در گام بعد، باید گذار از حالت $\{q_1, q_2\}$ برای نماد b را مشخص کنیم. در nfa، از q_1 هیچ گذاری برای b وجود ندارد و از q_2 یک گذار به q_0. بنابراین، اجتماع گذراها از $\{q_1, q_2\}$ برای b برابر با $\{q_0\}$ است.



قضیه: فرض کنید L_N زبانی است که پذیرنده‌ی متناهی غیرقطعی

$$M_N = (Q_N, \Sigma, \delta_N, q_0, F_N)$$

آن را می‌پذیرد. آنگاه یک پذیرنده متناهی قطعی

$$M_D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$$

وجود دارد به طوری‌که

$$L_N = L(M_D)$$